

ICP - 4
Tejaswini Tatikonda – 700762235

GitHub link: <https://github.com/tejaswini22350/Assignment-4.git>

Video link: https://drive.google.com/file/d/1t_uWLFw_98zjBPk8hHS85ukz-b9q_9ou/view?usp=share_link

ICP_Basics in Keras

Use Case Description:

Predicting the diabetes disease

Programming elements:

Keras Basics

In class programming:

1. Use the use case in the class:
 - a. Add more Dense layers to the existing code and check how the accuracy changes.
2. Change the data source to Breast Cancer dataset * available in the source code folder and make required changes. Report accuracy of the model.
3. Normalize the data before feeding the data to the model and check how the normalization change your accuracy (code given below).

```
from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()
```

Breast Cancer dataset is designated to predict if a patient has Malignant (M) or Benign = B cancer

In class programming:

Use Image Classification on the hand written digits data set (mnist)

1. 2. 3. 4. Plot the loss and accuracy for both training data and validation data using the history object in the source code.

Plot one of the images in the test data, and then do inferencing to check what is the prediction of the model

on that single image.

We had used 2 hidden layers and Relu activation. Try to change the number of hidden layer and the

activation to tanh or sigmoid and see what happens.

Run the same code without scaling the images and check the performance?

Evaluation Criteria:

1. Completeness of Features
2. Code Quality (https://en.wikipedia.org/wiki/Best_coding_practices)
3. Time
4. Feedback Submission

Note: Cheating, plagiarism, disruptive behavior and other forms of unacceptable conduct are subject to strong sanctions in accordance with university policy.

```
# 1. Import required libraries
import keras
import pandas
from keras.models import Sequential # For creating neural network model
from tensorflow.keras.layers import Dense # For adding dense layers
from sklearn.model_selection import train_test_split # For splitting dataset
import pandas as pd
import numpy as np

# 2. Load and prepare the dataset
dataset = pd.read_csv('diabetes.csv', header=None).values
# Reads the diabetes dataset and converts it to numpy array
# The dataset has 8 input features and 1 target variable (diabetes: yes/no)

# 3. Split the dataset into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(
    dataset[:,0:8], # Input features (first 8 columns)
    dataset[:,8],   # Target variable (last column)
    test_size=0.25, # 25% of data for testing
    random_state=87 # For reproducibility
)

# 4. Set random seed for reproducibility
np.random.seed(123)

# 5. Create the neural network model
my_first_nn = Sequential()

# 6. Add layers to the model
# First dense layer
my_first_nn.add(Dense(20, # 20 neurons
                      input_dim=8, # 8 input features
                      activation='relu')) # ReLU activation function
```

```

# Second dense layer
my_first_nn.add(Dense(4,          # 4 neurons
                      activation='relu')) # ReLU activation function

# Output layer
my_first_nn.add(Dense(1,          # 1 neuron (binary output)
                      activation='sigmoid')) # Sigmoid for binary classification

# 7. Compile the model
my_first_nn.compile(
    loss='binary_crossentropy', # Loss function for binary classification
    optimizer='adam',           # Adam optimizer
    metrics=['acc']             # Track accuracy during training
)

# 8. Train the model
my_first_nn_fitted = my_first_nn.fit(
    X_train, Y_train,           # Training data
    epochs=25,                  # Number of training iterations
    initial_epoch=0             # Starting epoch
)

# 9. Display model summary
print(my_first_nn.summary())

# 10. Evaluate model on test data
print(my_first_nn.evaluate(X_test, Y_test))

```

```

Epoch 1/25
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
18/18 ----- 1s 3ms/step - acc: 0.6427 - loss: 6.5889
Epoch 2/25
18/18 ----- 0s 3ms/step - acc: 0.6580 - loss: 4.2612
Epoch 3/25
18/18 ----- 0s 3ms/step - acc: 0.6187 - loss: 2.1965
Epoch 4/25
18/18 ----- 0s 3ms/step - acc: 0.4351 - loss: 1.2542

```

Epoch 5/25
18/18  0s 4ms/step - acc: 0.3424 - loss: 0.8129
Epoch 6/25
18/18  0s 3ms/step - acc: 0.3436 - loss: 0.7889
Epoch 7/25
18/18  0s 4ms/step - acc: 0.3374 - loss: 0.7290
Epoch 8/25
18/18  0s 5ms/step - acc: 0.3298 - loss: 0.7276
Epoch 9/25
18/18  0s 5ms/step - acc: 0.3211 - loss: 0.7299
Epoch 10/25
18/18  0s 5ms/step - acc: 0.5941 - loss: 0.7099
Epoch 11/25
18/18  0s 5ms/step - acc: 0.6834 - loss: 0.7038
Epoch 12/25
18/18  0s 6ms/step - acc: 0.6441 - loss: 0.7047
Epoch 13/25
18/18  0s 5ms/step - acc: 0.6738 - loss: 0.6960
Epoch 14/25
18/18  0s 5ms/step - acc: 0.6597 - loss: 0.6885
Epoch 15/25
18/18  0s 4ms/step - acc: 0.7015 - loss: 0.7020
Epoch 16/25
18/18  0s 5ms/step - acc: 0.6574 - loss: 0.6900
Epoch 17/25
18/18  0s 5ms/step - acc: 0.6562 - loss: 0.6794
Epoch 18/25
18/18  0s 5ms/step - acc: 0.6406 - loss: 0.6764
Epoch 19/25
18/18  0s 6ms/step - acc: 0.6572 - loss: 0.6773
Epoch 20/25
18/18  0s 6ms/step - acc: 0.6894 - loss: 0.6720
Epoch 21/25
18/18  0s 5ms/step - acc: 0.6667 - loss: 0.6698
Epoch 22/25
18/18  0s 5ms/step - acc: 0.6544 - loss: 0.6695
Epoch 23/25
18/18  0s 3ms/step - acc: 0.6566 - loss: 0.6655
Epoch 24/25
18/18  0s 3ms/step - acc: 0.6844 - loss: 0.6615

Epoch 25/25

18/18  0s 3ms/step - acc: 0.6263 - loss: 0.6718

Model: "sequential_20"

Layer (type)	Output Shape	Param #
dense_50 (Dense)	(None, 20)	180
dense_51 (Dense)	(None, 4)	84
dense_52 (Dense)	(None, 1)	5

Total params: 809 (3.16 KB)

Trainable params: 269 (1.05 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 540 (2.11 KB)

None

6/6  0s 8ms/step - acc: 0.6463 - loss: 0.6651

[0.6697693467140198, 0.6197916865348816]

```
# 1. Import required libraries
import keras
import pandas as pd
import numpy as np
from keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_breast_cancer # Built-in breast cancer dataset
from sklearn.model_selection import train_test_split

# 2. Load the breast cancer dataset
cancer_data = load_breast_cancer()
# This dataset includes:
# - 30 features (measurements of breast cancer tumors)
# - Binary target (malignant or benign)

# 3. Split the dataset into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(
    cancer_data.data,      # Features matrix
    cancer_data.target,    # Target variable
    test_size=0.25,       # 25% for testing
    random_state=87       # For reproducibility
)

# 4. Set random seed for reproducibility
np.random.seed(155)

# 5. Create the neural network model
my_nn = Sequential()

# 6. Add layers to the model
# First (and only hidden) layer
my_nn.add(Dense(20,      # 20 neurons
               input_dim=30, # 30 input features
               activation='relu')) # ReLU activation
```

```

# Output layer
my_nn.add(Dense(1,          # 1 neuron (binary output)
                activation='sigmoid')) # Sigmoid for binary classification

# 7. Compile the model
my_nn.compile(
    loss='binary_crossentropy', # Loss function for binary classification
    optimizer='adam',          # Adam optimizer
    metrics=['acc']             # Track accuracy
)

# 8. Train the model
my_nn_fitted = my_nn.fit(
    X_train, Y_train, # Training data
    epochs=100,       # Number of training iterations
    initial_epoch=0    # Starting epoch
)

# 9. Display model summary
print(my_nn.summary())

# 10. Evaluate model on test data
print(my_nn.evaluate(X_test, Y_test))

```

Epoch 1/100		
14/14		1s 4ms/step - acc: 0.3674 - loss: 62.8302
Epoch 2/100		
14/14		0s 4ms/step - acc: 0.3918 - loss: 29.9527
Epoch 3/100		
14/14		0s 5ms/step - acc: 0.5508 - loss: 3.5090
Epoch 4/100		
14/14		0s 5ms/step - acc: 0.8183 - loss: 2.0928
Epoch 5/100		
14/14		0s 4ms/step - acc: 0.8287 - loss: 0.5335
Epoch 6/100		
14/14		0s 4ms/step - acc: 0.9155 - loss: 0.3607
Epoch 7/100		
14/14		0s 4ms/step - acc: 0.9321 - loss: 0.2122
Epoch 8/100		
14/14		0s 4ms/step - acc: 0.9215 - loss: 0.2408
Epoch 9/100		
14/14		0s 4ms/step - acc: 0.9427 - loss: 0.1882
Epoch 10/100		
14/14		0s 4ms/step - acc: 0.9289 - loss: 0.2042
Epoch 11/100		
14/14		0s 4ms/step - acc: 0.9348 - loss: 0.1977
Epoch 12/100		
14/14		0s 5ms/step - acc: 0.9259 - loss: 0.2584
Epoch 13/100		
14/14		0s 4ms/step - acc: 0.9257 - loss: 0.2140
Epoch 14/100		
14/14		0s 4ms/step - acc: 0.9296 - loss: 0.1866
Epoch 15/100		
14/14		0s 4ms/step - acc: 0.9355 - loss: 0.1979
Epoch 16/100		
14/14		0s 4ms/step - acc: 0.9278 - loss: 0.2391
Epoch 17/100		
14/14		0s 4ms/step - acc: 0.9380 - loss: 0.1967
Epoch 18/100		
14/14		0s 4ms/step - acc: 0.9157 - loss: 0.2157
Epoch 19/100		
14/14		0s 3ms/step - acc: 0.9273 - loss: 0.2326
Epoch 20/100		
14/14		0s 4ms/step - acc: 0.9345 - loss: 0.1811

```
Epoch 21/100
14/14  0s 5ms/step - acc: 0.9147 - loss: 0.2358
Epoch 22/100
14/14  0s 5ms/step - acc: 0.9388 - loss: 0.2061
cell output actions
14/14  0s 4ms/step - acc: 0.9274 - loss: 0.1989
Epoch 24/100
14/14  0s 4ms/step - acc: 0.9222 - loss: 0.2345
Epoch 25/100
14/14  0s 4ms/step - acc: 0.9251 - loss: 0.1969
Epoch 26/100
14/14  0s 6ms/step - acc: 0.9228 - loss: 0.2098
Epoch 27/100
14/14  0s 6ms/step - acc: 0.9353 - loss: 0.1895
Epoch 28/100
14/14  0s 5ms/step - acc: 0.9379 - loss: 0.1888
Epoch 29/100
14/14  0s 6ms/step - acc: 0.9222 - loss: 0.1813
Epoch 30/100
14/14  0s 7ms/step - acc: 0.9267 - loss: 0.2232
Epoch 31/100
14/14  0s 5ms/step - acc: 0.9480 - loss: 0.1674
Epoch 32/100
14/14  0s 5ms/step - acc: 0.9451 - loss: 0.1701
Epoch 33/100
14/14  0s 5ms/step - acc: 0.9278 - loss: 0.2438
Epoch 34/100
14/14  0s 5ms/step - acc: 0.9504 - loss: 0.1484
Epoch 35/100
14/14  0s 6ms/step - acc: 0.9353 - loss: 0.1692
Epoch 36/100
14/14  0s 8ms/step - acc: 0.9084 - loss: 0.2730
Epoch 37/100
14/14  0s 6ms/step - acc: 0.9339 - loss: 0.1783
Epoch 38/100
14/14  0s 6ms/step - acc: 0.9113 - loss: 0.2412
Epoch 39/100
14/14  0s 5ms/step - acc: 0.9381 - loss: 0.1980
Epoch 40/100
14/14  0s 5ms/step - acc: 0.9319 - loss: 0.1666
```

```
Epoch 41/100
14/14  0s 6ms/step - acc: 0.9159 - loss: 0.1691
Epoch 42/100
14/14  0s 6ms/step - acc: 0.9323 - loss: 0.1781
Epoch 43/100
14/14  0s 4ms/step - acc: 0.9198 - loss: 0.1870
Epoch 44/100
14/14  0s 4ms/step - acc: 0.9256 - loss: 0.2214
Epoch 45/100
14/14  0s 4ms/step - acc: 0.9414 - loss: 0.1663
Epoch 46/100
14/14  0s 4ms/step - acc: 0.9419 - loss: 0.1732
Epoch 47/100
14/14  0s 4ms/step - acc: 0.9206 - loss: 0.1905
Epoch 48/100
14/14  0s 4ms/step - acc: 0.9303 - loss: 0.2111
Epoch 49/100
14/14  0s 5ms/step - acc: 0.9033 - loss: 0.3797
Epoch 50/100
14/14  0s 5ms/step - acc: 0.9262 - loss: 0.1861
Epoch 51/100
14/14  0s 3ms/step - acc: 0.9407 - loss: 0.1542
Epoch 52/100
14/14  0s 4ms/step - acc: 0.9501 - loss: 0.1696
Epoch 53/100
14/14  0s 4ms/step - acc: 0.9064 - loss: 0.2302
Epoch 54/100
14/14  0s 4ms/step - acc: 0.9201 - loss: 0.1731
Epoch 55/100
14/14  0s 4ms/step - acc: 0.9309 - loss: 0.1998
Epoch 56/100
14/14  0s 4ms/step - acc: 0.9036 - loss: 0.1973
Epoch 57/100
14/14  0s 4ms/step - acc: 0.9260 - loss: 0.2002
Epoch 58/100
14/14  0s 4ms/step - acc: 0.9129 - loss: 0.2099
Epoch 59/100
14/14  0s 4ms/step - acc: 0.9285 - loss: 0.2130
Epoch 60/100
14/14  0s 4ms/step - acc: 0.9194 - loss: 0.2447
```

Epoch 61/100
14/14  0s 4ms/step - acc: 0.9099 - loss: 0.2207
Epoch 62/100
14/14  0s 4ms/step - acc: 0.9201 - loss: 0.2189
Epoch 63/100
14/14  0s 4ms/step - acc: 0.9432 - loss: 0.1424
Epoch 64/100
14/14  0s 4ms/step - acc: 0.9380 - loss: 0.1572
Epoch 65/100
14/14  0s 5ms/step - acc: 0.9046 - loss: 0.2075
Epoch 66/100
14/14  0s 4ms/step - acc: 0.9305 - loss: 0.1954
Epoch 67/100
14/14  0s 4ms/step - acc: 0.9181 - loss: 0.2170
Epoch 68/100
14/14  0s 4ms/step - acc: 0.9134 - loss: 0.2252
Epoch 69/100
14/14  0s 4ms/step - acc: 0.9172 - loss: 0.1904
Epoch 70/100
14/14  0s 4ms/step - acc: 0.9300 - loss: 0.1639
Epoch 71/100
14/14  0s 4ms/step - acc: 0.9366 - loss: 0.1400
Epoch 72/100
14/14  0s 4ms/step - acc: 0.9201 - loss: 0.1802
Epoch 73/100
14/14  0s 5ms/step - acc: 0.9164 - loss: 0.1702
Epoch 74/100
14/14  0s 4ms/step - acc: 0.9321 - loss: 0.1987
Epoch 75/100
14/14  0s 4ms/step - acc: 0.9414 - loss: 0.1783
Epoch 76/100
14/14  0s 4ms/step - acc: 0.9149 - loss: 0.2154
Epoch 77/100
14/14  0s 4ms/step - acc: 0.9275 - loss: 0.2064
Epoch 78/100
14/14  0s 4ms/step - acc: 0.9369 - loss: 0.1540
Epoch 79/100
14/14  0s 4ms/step - acc: 0.9218 - loss: 0.1561

```
Epoch 80/100
14/14  0s 4ms/step - acc: 0.9224 - loss: 0.2281
Epoch 81/100
14/14  0s 4ms/step - acc: 0.9353 - loss: 0.1480
Epoch 82/100
14/14  0s 4ms/step - acc: 0.9229 - loss: 0.1887
Epoch 83/100
14/14  0s 4ms/step - acc: 0.9370 - loss: 0.1552
Epoch 84/100
14/14  0s 5ms/step - acc: 0.9320 - loss: 0.1689
Epoch 85/100
14/14  0s 4ms/step - acc: 0.9394 - loss: 0.1453
Epoch 86/100
14/14  0s 4ms/step - acc: 0.9371 - loss: 0.1534
Epoch 87/100
14/14  0s 4ms/step - acc: 0.9221 - loss: 0.1871
Epoch 88/100
14/14  0s 4ms/step - acc: 0.9282 - loss: 0.1620
Epoch 89/100
14/14  0s 4ms/step - acc: 0.9196 - loss: 0.2089
Epoch 90/100
14/14  0s 4ms/step - acc: 0.9457 - loss: 0.1286
Epoch 91/100
14/14  0s 4ms/step - acc: 0.9386 - loss: 0.1676
Epoch 92/100
14/14  0s 4ms/step - acc: 0.9201 - loss: 0.1699
Epoch 93/100
14/14  0s 4ms/step - acc: 0.9309 - loss: 0.1598
Epoch 94/100
14/14  0s 4ms/step - acc: 0.9181 - loss: 0.1991
Epoch 95/100
14/14  0s 4ms/step - acc: 0.9378 - loss: 0.1359
Epoch 96/100
14/14  0s 4ms/step - acc: 0.9453 - loss: 0.1414
Epoch 97/100
14/14  0s 5ms/step - acc: 0.9378 - loss: 0.1552
Epoch 98/100
14/14  0s 4ms/step - acc: 0.9102 - loss: 0.2219
Epoch 99/100
14/14  0s 4ms/step - acc: 0.9473 - loss: 0.1336
```

Epoch 89/100
 14/14 ————— 0s 4ms/step - acc: 0.9196 - loss: 0.2089
 Epoch 90/100
 14/14 ————— 0s 4ms/step - acc: 0.9457 - loss: 0.1286
 Epoch 91/100
 14/14 ————— 0s 4ms/step - acc: 0.9386 - loss: 0.1676
 Epoch 92/100
 14/14 ————— 0s 4ms/step - acc: 0.9201 - loss: 0.1699
 Epoch 93/100
 14/14 ————— 0s 4ms/step - acc: 0.9309 - loss: 0.1598
 Epoch 94/100
 14/14 ————— 0s 4ms/step - acc: 0.9181 - loss: 0.1991
 Epoch 95/100
 14/14 ————— 0s 4ms/step - acc: 0.9378 - loss: 0.1359
 Epoch 96/100
 14/14 ————— 0s 4ms/step - acc: 0.9453 - loss: 0.1414
 Epoch 97/100
 14/14 ————— 0s 5ms/step - acc: 0.9378 - loss: 0.1552
 Epoch 98/100
 14/14 ————— 0s 4ms/step - acc: 0.9102 - loss: 0.2219
 Epoch 99/100
 14/14 ————— 0s 4ms/step - acc: 0.9473 - loss: 0.1336
 Epoch 100/100
 14/14 ————— 0s 4ms/step - acc: 0.9535 - loss: 0.1132

Model: "sequential_21"

Layer (type)	Output Shape	Param #
dense_53 (Dense)	(None, 20)	620
dense_54 (Dense)	(None, 1)	21

Total params: 1,925 (7.52 KB)
 Trainable params: 641 (2.50 KB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 1,284 (5.02 KB)

None

5/5 ————— 0s 7ms/step - acc: 0.8997 - loss: 0.4367
 [0.3489443361759186, 0.9020978808403015]

```

# 1. Import necessary libraries
from sklearn.preprocessing import StandardScaler # For data normalization
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# 2. Load the breast cancer dataset
data = load_breast_cancer()
X = data.data # Features
y = data.target # Target labels (malignant/benign)

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, # 20% for testing
    random_state=42 # For reproducibility
)

# 4. Normalize the data using StandardScaler
sc = StandardScaler()
# Fit and transform training data
X_train_normalized = sc.fit_transform(X_train)
# Transform test data (using parameters from training data)
X_test_normalized = sc.transform(X_test)

# 5. Create neural network model
model_normalized = Sequential()

# 6. Add layers with increasing complexity
# First hidden layer
model_normalized.add(Dense(32, # 32 neurons
                             activation='relu', # ReLU activation
                             input_shape=(X_train.shape[1],))) # Input shape based on feature count

```

```

# Second hidden layer
model_normalized.add(Dense(64,          # 64 neurons
                           activation='relu'))

# Third hidden layer
model_normalized.add(Dense(128,         # 128 neurons
                           activation='relu'))

# Output layer
model_normalized.add(Dense(1,           # Binary output
                           activation='sigmoid'))

# 7. Compile the model
model_normalized.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# 8. Train the model
model_normalized.fit(
    X_train_normalized, y_train,
    epochs=10,                # Number of training iterations
    batch_size=32,            # Samples per gradient update
    validation_data=(X_test_normalized, y_test) # For monitoring validation performance
)

# 9. Evaluate model and print accuracy
accuracy_normalized = model_normalized.evaluate(X_test_normalized, y_test)[1]
print("Accuracy with normalization:", accuracy_normalized)

```

```

Epoch 1/10
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
15/15 ----- 2s 22ms/step - accuracy: 0.6927 - loss: 0.5724 - val_accuracy: 0.8947 - val_loss: 0.3247
Epoch 2/10
15/15 ----- 0s 7ms/step - accuracy: 0.9339 - loss: 0.2862 - val_accuracy: 0.9561 - val_loss: 0.1577
Epoch 3/10
15/15 ----- 0s 10ms/step - accuracy: 0.9647 - loss: 0.1622 - val_accuracy: 0.9649 - val_loss: 0.1054
Epoch 4/10
15/15 ----- 0s 10ms/step - accuracy: 0.9811 - loss: 0.1006 - val_accuracy: 0.9649 - val_loss: 0.0854
Epoch 5/10
15/15 ----- 0s 10ms/step - accuracy: 0.9783 - loss: 0.0847 - val_accuracy: 0.9649 - val_loss: 0.0813
Epoch 6/10
15/15 ----- 0s 12ms/step - accuracy: 0.9838 - loss: 0.0837 - val_accuracy: 0.9561 - val_loss: 0.0870
Epoch 7/10
15/15 ----- 0s 12ms/step - accuracy: 0.9901 - loss: 0.0436 - val_accuracy: 0.9474 - val_loss: 0.0858
Epoch 8/10
15/15 ----- 0s 14ms/step - accuracy: 0.9905 - loss: 0.0452 - val_accuracy: 0.9474 - val_loss: 0.0919
Epoch 9/10
15/15 ----- 0s 10ms/step - accuracy: 0.9928 - loss: 0.0335 - val_accuracy: 0.9474 - val_loss: 0.0893
Epoch 10/10
15/15 ----- 0s 13ms/step - accuracy: 0.9927 - loss: 0.0314 - val_accuracy: 0.9474 - val_loss: 0.0908
4/4 ----- 0s 17ms/step - accuracy: 0.9467 - loss: 0.1063
Accuracy with normalization: 0.9473684430122375

```

```

# 1. Import required libraries
import keras
from keras.datasets import mnist # MNIST dataset of handwritten digits
from keras.models import Sequential
from keras.layers import Dense, Dropout # Dropout for regularization
import matplotlib.pyplot as plt # For plotting results

# 2. Load and prepare MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

# 3. Normalize pixel values to range [0,1]
train_images = train_images.astype('float32') / 255
test_images = test_images.astype('float32') / 255

# 4. Define number of classes (digits 0-9)
num_classes = 10

# 5. Convert labels to one-hot encoded format
train_labels = keras.utils.to_categorical(train_labels, num_classes)
test_labels = keras.utils.to_categorical(test_labels, num_classes)

# 6. Create neural network model
model = Sequential()

# 7. Add layers to the model
# First dense layer
model.add(Dense(512, # 512 neurons
                 activation='relu', # ReLU activation
                 input_shape=(784,))) # Input shape (flattened 28x28 images)
model.add(Dropout(0.2)) # 20% dropout for regularization

# Second dense layer
model.add(Dense(512, # 512 neurons
                 activation='relu'))
model.add(Dropout(0.2)) # 20% dropout

```

```

# Output layer
model.add(Dense(num_classes,          # 10 neurons (one per digit)
               activation='softmax'))  # Softmax for multiclass classification

# 8. Compile the model
model.compile(
    loss='categorical_crossentropy',  # Loss for multiclass classification
    optimizer='adam',                 # Adam optimizer
    metrics=['accuracy']              # Track accuracy
)

# 9. Train the model
history = model.fit(
    train_images.reshape(-1, 784),     # Flatten 28x28 images to 784-pixel vector
    train_labels,
    validation_data=(test_images.reshape(-1, 784), test_labels),
    epochs=20,                         # Number of training iterations
    batch_size=128                     # Samples per gradient update
)

# 10. Plot training results
plt.figure(figsize=(10, 5))

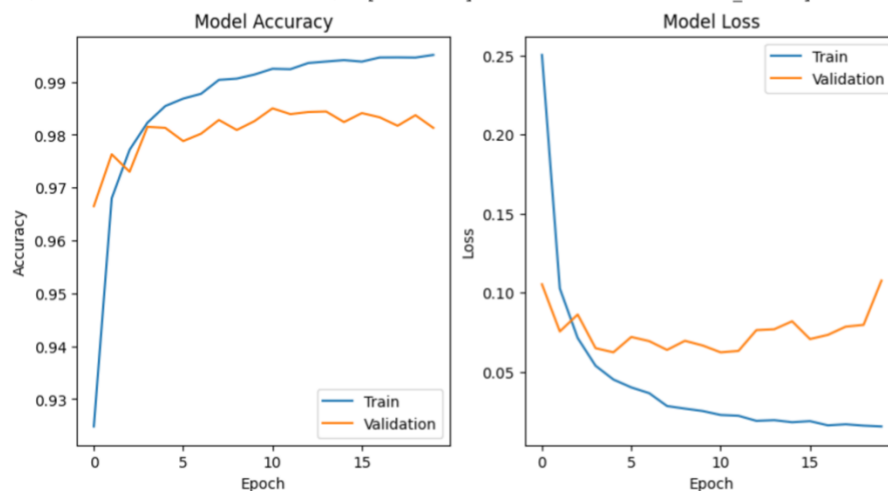
# Plot accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='lower right')

```

```
# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper right')

plt.show()
```

```
Epoch 1/20
469/469 ————— 12s 22ms/step - accuracy: 0.8630 - loss: 0.4508 - val_accuracy: 0.9665 - val_loss: 0.0820
Epoch 2/20
469/469 ————— 20s 22ms/step - accuracy: 0.9664 - loss: 0.1072 - val_accuracy: 0.9763 - val_loss: 0.0708
Epoch 3/20
469/469 ————— 9s 20ms/step - accuracy: 0.9768 - loss: 0.0719 - val_accuracy: 0.9730 - val_loss: 0.0734
Epoch 4/20
469/469 ————— 11s 21ms/step - accuracy: 0.9823 - loss: 0.0529 - val_accuracy: 0.9815 - val_loss: 0.0786
Epoch 5/20
469/469 ————— 11s 23ms/step - accuracy: 0.9864 - loss: 0.0418 - val_accuracy: 0.9813 - val_loss: 0.0797
Epoch 6/20
469/469 ————— 20s 22ms/step - accuracy: 0.9880 - loss: 0.0375 - val_accuracy: 0.9788 - val_loss: 0.0786
Epoch 7/20
469/469 ————— 12s 25ms/step - accuracy: 0.9878 - loss: 0.0356 - val_accuracy: 0.9802 - val_loss: 0.0777
Epoch 8/20
469/469 ————— 9s 20ms/step - accuracy: 0.9912 - loss: 0.0260 - val_accuracy: 0.9828 - val_loss: 0.0777
Epoch 9/20
469/469 ————— 10s 22ms/step - accuracy: 0.9908 - loss: 0.0257 - val_accuracy: 0.9809 - val_loss: 0.0777
Epoch 10/20
469/469 ————— 11s 22ms/step - accuracy: 0.9914 - loss: 0.0255 - val_accuracy: 0.9826 - val_loss: 0.0777
Epoch 11/20
469/469 ————— 20s 22ms/step - accuracy: 0.9929 - loss: 0.0216 - val_accuracy: 0.9850 - val_loss: 0.0777
Epoch 12/20
469/469 ————— 19s 20ms/step - accuracy: 0.9927 - loss: 0.0211 - val_accuracy: 0.9839 - val_loss: 0.0777
Epoch 13/20
469/469 ————— 11s 22ms/step - accuracy: 0.9943 - loss: 0.0173 - val_accuracy: 0.9843 - val_loss: 0.0777
Epoch 14/20
469/469 ————— 10s 22ms/step - accuracy: 0.9942 - loss: 0.0182 - val_accuracy: 0.9844 - val_loss: 0.0777
Epoch 15/20
469/469 ————— 11s 23ms/step - accuracy: 0.9947 - loss: 0.0160 - val_accuracy: 0.9824 - val_loss: 0.0820
Epoch 16/20
469/469 ————— 20s 22ms/step - accuracy: 0.9933 - loss: 0.0197 - val_accuracy: 0.9841 - val_loss: 0.0708
Epoch 17/20
469/469 ————— 10s 20ms/step - accuracy: 0.9941 - loss: 0.0185 - val_accuracy: 0.9833 - val_loss: 0.0734
Epoch 18/20
469/469 ————— 11s 23ms/step - accuracy: 0.9951 - loss: 0.0163 - val_accuracy: 0.9817 - val_loss: 0.0786
Epoch 19/20
469/469 ————— 11s 22ms/step - accuracy: 0.9952 - loss: 0.0147 - val_accuracy: 0.9837 - val_loss: 0.0797
Epoch 20/20
469/469 ————— 11s 22ms/step - accuracy: 0.9951 - loss: 0.0158 - val_accuracy: 0.9813 - val_loss: 0.1077
```



```
# 1. Import required libraries
import keras
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense, Dropout
import matplotlib.pyplot as plt
import numpy as np

# 2. Load and prepare MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

# 3. Normalize pixel values to range [0,1]
train_images = train_images.astype('float32') / 255 # Convert to float and scale
test_images = test_images.astype('float32') / 255

# 4. Set up classification parameters
num_classes = 10 # 10 digits (0-9)

# 5. Convert labels to one-hot encoded format
train_labels = keras.utils.to_categorical(train_labels, num_classes)
test_labels = keras.utils.to_categorical(test_labels, num_classes)

# 6. Create neural network model
model = Sequential()
```

```

# 7. Add layers to the model
# First dense layer
model.add(Dense(512,                # 512 neurons
                activation='relu',   # ReLU activation
                input_shape=(784,))) # Input shape (flattened 28x28 images)
model.add(Dropout(0.2))             # 20% dropout for regularization

# Second dense layer
model.add(Dense(512,                # 512 neurons
                activation='relu'))
model.add(Dropout(0.2))             # 20% dropout

# Output layer
model.add(Dense(num_classes,        # 10 neurons (one per digit)
                activation='softmax')) # Softmax for multiclass classification

# 8. Compile the model
model.compile(
    loss='categorical_crossentropy', # Loss for multiclass classification
    optimizer='adam',               # Adam optimizer
    metrics=['accuracy']            # Track accuracy
)

# 9. Train the model
model.fit(
    train_images.reshape(-1, 784),   # Flatten images to 784-dim vector
    train_labels,
    validation_data=(test_images.reshape(-1, 784), test_labels),
    epochs=20,                       # Number of training iterations
    batch_size=128                   # Samples per gradient update
)

# 10. Visualize a test image
plt.imshow(test_images[0], cmap='gray') # Display first test image
plt.show()

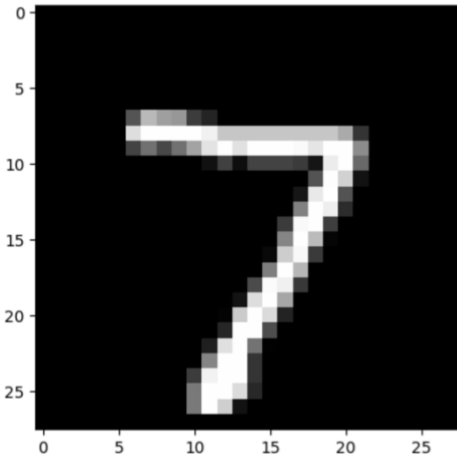
```

```
# 11. Make prediction on the displayed image
prediction = model.predict(test_images[0].reshape(1, -1)) # Reshape for model input
print('Model prediction:', np.argmax(prediction)) # Get predicted digit
```

```
Epoch 1/20
469/469 ————— 12s 23ms/step - accuracy: 0.8651 - loss: 0.4507 - val_accuracy: 0.9667 - val_loss: 0.1056
Epoch 2/20
469/469 ————— 20s 22ms/step - accuracy: 0.9670 - loss: 0.1093 - val_accuracy: 0.9779 - val_loss: 0.0751
Epoch 3/20
469/469 ————— 10s 21ms/step - accuracy: 0.9759 - loss: 0.0754 - val_accuracy: 0.9771 - val_loss: 0.0759
Epoch 4/20
469/469 ————— 10s 20ms/step - accuracy: 0.9833 - loss: 0.0535 - val_accuracy: 0.9777 - val_loss: 0.0733
Epoch 5/20
469/469 ————— 11s 22ms/step - accuracy: 0.9862 - loss: 0.0426 - val_accuracy: 0.9807 - val_loss: 0.0621
Epoch 6/20
469/469 ————— 10s 22ms/step - accuracy: 0.9878 - loss: 0.0372 - val_accuracy: 0.9799 - val_loss: 0.0666
Epoch 7/20
469/469 ————— 11s 23ms/step - accuracy: 0.9891 - loss: 0.0346 - val_accuracy: 0.9811 - val_loss: 0.0682
Epoch 8/20
469/469 ————— 19s 20ms/step - accuracy: 0.9902 - loss: 0.0303 - val_accuracy: 0.9816 - val_loss: 0.0685
Epoch 9/20
469/469 ————— 11s 22ms/step - accuracy: 0.9909 - loss: 0.0287 - val_accuracy: 0.9839 - val_loss: 0.0615
Epoch 10/20
469/469 ————— 21s 22ms/step - accuracy: 0.9918 - loss: 0.0234 - val_accuracy: 0.9807 - val_loss: 0.0776
Epoch 11/20
469/469 ————— 11s 23ms/step - accuracy: 0.9929 - loss: 0.0219 - val_accuracy: 0.9827 - val_loss: 0.0680
Epoch 12/20
469/469 ————— 11s 23ms/step - accuracy: 0.9924 - loss: 0.0235 - val_accuracy: 0.9803 - val_loss: 0.0787
Epoch 13/20
469/469 ————— 20s 22ms/step - accuracy: 0.9931 - loss: 0.0209 - val_accuracy: 0.9819 - val_loss: 0.0757
Epoch 14/20
469/469 ————— 21s 23ms/step - accuracy: 0.9950 - loss: 0.0156 - val_accuracy: 0.9827 - val_loss: 0.0776
Epoch 15/20
469/469 ————— 21s 23ms/step - accuracy: 0.9922 - loss: 0.0224 - val_accuracy: 0.9820 - val_loss: 0.0772
Epoch 16/20
469/469 ————— 9s 20ms/step - accuracy: 0.9942 - loss: 0.0171 - val_accuracy: 0.9818 - val_loss: 0.0790
Epoch 17/20
469/469 ————— 10s 22ms/step - accuracy: 0.9942 - loss: 0.0161 - val_accuracy: 0.9822 - val_loss: 0.0799
Epoch 18/20
469/469 ————— 20s 22ms/step - accuracy: 0.9955 - loss: 0.0146 - val_accuracy: 0.9812 - val_loss: 0.0862
Epoch 19/20
```

✓ 4m 39s completed at 10:04 PM

```
Epoch 19/20
469/469 ————— 10s 22ms/step - accuracy: 0.9954 - loss: 0.0139 - val_accuracy: 0.9832 - val_loss: 0.0773
Epoch 20/20
469/469 ————— 9s 20ms/step - accuracy: 0.9957 - loss: 0.0130 - val_accuracy: 0.9834 - val_loss: 0.0807
```



```
1/1 ————— 0s 106ms/step
Model prediction: 7
```

```

import keras
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense, Dropout
import matplotlib.pyplot as plt
import numpy as np

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.astype('float32') / 255
test_images = test_images.astype('float32') / 255

num_classes = 10
train_labels = keras.utils.to_categorical(train_labels, num_classes)
test_labels = keras.utils.to_categorical(test_labels, num_classes)

models = []

model = Sequential()
model.add(Dense(512, activation='tanh', input_shape=(784,)))
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
models.append(('1 hidden layer with tanh', model))

model = Sequential()
model.add(Dense(512, activation='sigmoid', input_shape=(784,)))
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
models.append(('1 hidden layer with sigmoid', model))

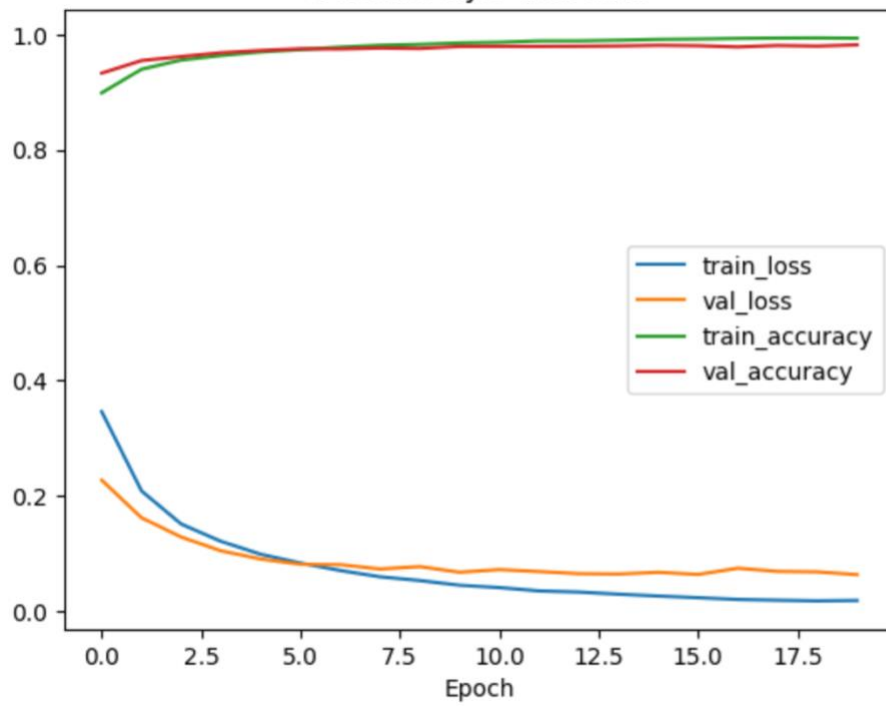
model = Sequential()
model.add(Dense(512, activation='tanh', input_shape=(784,)))
model.add(Dropout(0.2))
model.add(Dense(512, activation='tanh'))
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
models.append(('2 hidden layers with tanh', model))

model = Sequential()
model.add(Dense(512, activation='sigmoid', input_shape=(784,)))
model.add(Dropout(0.2))
model.add(Dense(512, activation='sigmoid'))
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
models.append(('2 hidden layers with sigmoid', model))

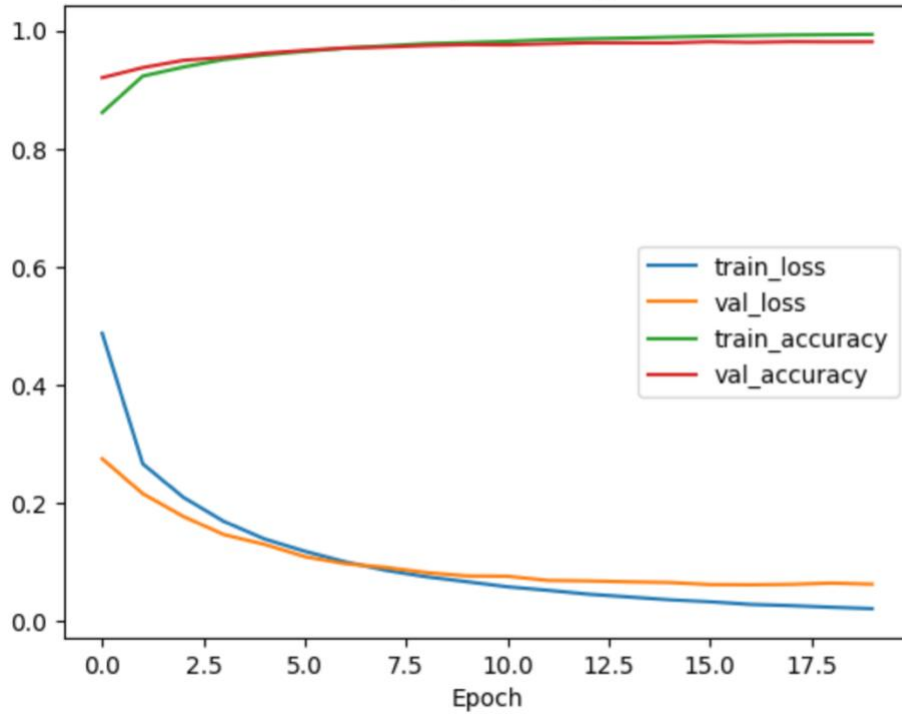
for name, model in models:
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    history = model.fit(train_images.reshape(-1, 784), train_labels, validation_data=(test_images.reshape(-1, 784), test_labels),
                        epochs=20, batch_size=128, verbose=0)
    plt.plot(history.history['loss'], label='train_loss')
    plt.plot(history.history['val_loss'], label='val_loss')
    plt.plot(history.history['accuracy'], label='train_accuracy')
    plt.plot(history.history['val_accuracy'], label='val_accuracy')
    plt.title(name)
    plt.xlabel('Epoch')
    plt.legend()
    plt.show()
    loss, accuracy = model.evaluate(test_images.reshape(-1, 784), test_labels, verbose=0)
    print('{} - Test loss: {:.4f}, Test accuracy: {:.4f}'.format(name, loss, accuracy))

```

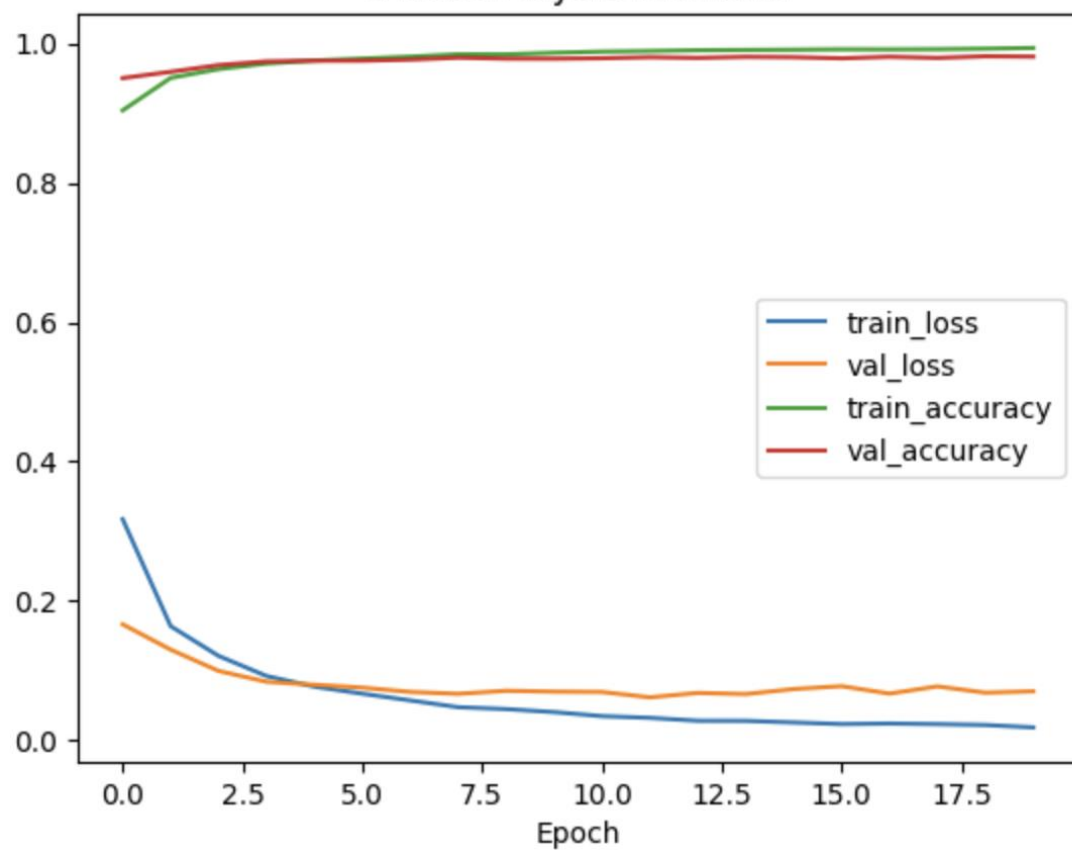
1 hidden layer with tanh

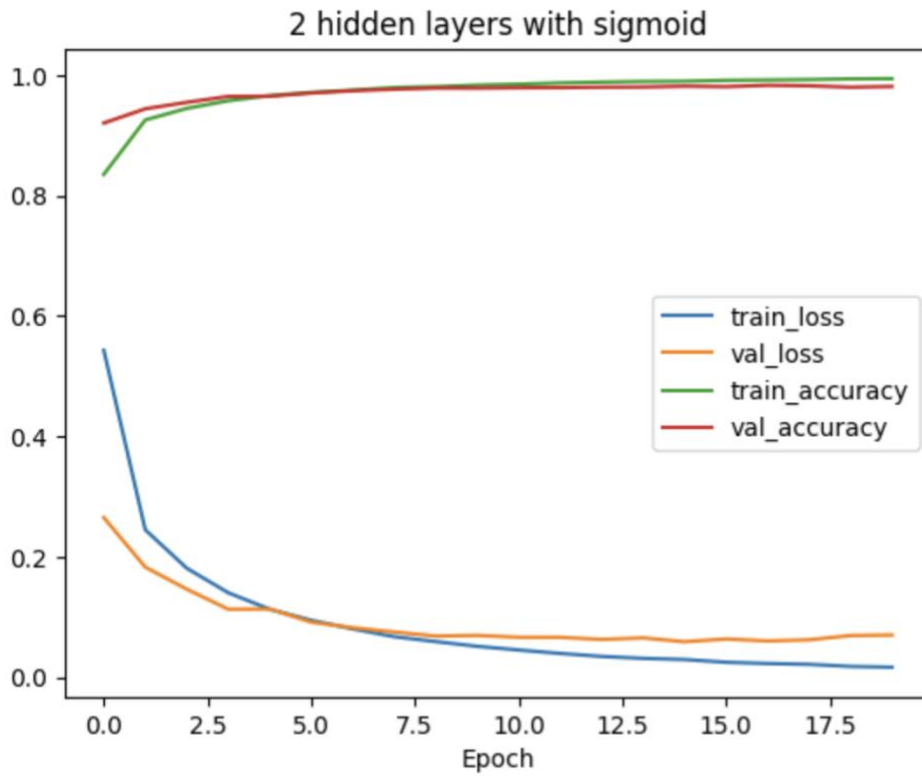


1 hidden layer with sigmoid



2 hidden layers with tanh





2 hidden layers with sigmoid - Test loss: 0.0699, Test accuracy: 0.9815
